

Learning to Race with PPO

How Policy Network Size Affects Performance, Convergence, and Training Time

Yagiz Tansu

Reinforcement Learning — AA 2025–26

Contents

1	Abstract	3
2	Introduction	4
2.1	Motivation	4
2.2	Goal	4
2.3	Research questions	4
3	Background	5
3.1	Reinforcement learning basics	5
3.2	Policy gradient methods	5
3.3	PPO (Proximal Policy Optimization)	5
3.4	Why this problem fits RL	5
4	Environment Design	6
4.1	Track generation	6
4.2	Car dynamics	6
4.3	Frenet coordinates	7
4.4	Observation space (14 numbers)	7
4.5	Action space (2 numbers)	7
4.6	Reward function	7
5	Training and Evaluation	9
5.1	Network architectures	9
5.2	Training procedure	9
5.3	Evaluation procedure	9
6	Experiments and Results	10
6.1	Learning curves	10
6.2	Final performance	10
6.3	Convergence in episodes	11
6.4	Convergence in wall-clock time	11
6.5	Policy exploration (sigma decay)	11
6.6	Summary table	13

7	Discussion	14
7.1	Network size vs performance	14
7.2	Network size vs training cost	14
7.3	Limitations	14
7.4	Reproducibility	14
8	Conclusion	15

1 Abstract

A reinforcement learning agent is trained to drive a car around a 2D racetrack in minimum time. The agent uses **Proximal Policy Optimization (PPO)** with a continuous Gaussian policy. The car is controlled through throttle/brake and steering.

The main question of this project is simple: **does a bigger neural network help?** Three policy network sizes are tested and three quantities are measured: (1) final lap performance, (2) how many episodes are needed to learn, and (3) real wall-clock training time.

The environment was custom-built with Gymnasium. It uses a kinematic bicycle model for physics and Frenet coordinates for the state. The agent sees its lateral offset, heading error, speed, and upcoming track curvature — but not its absolute position on the map.

The results show that all three network sizes learn to complete a full lap with similar final return (about 1139). Lap times differ only slightly; the medium network [64, 64] achieves the fastest lap. For convergence, [64, 64] needs the fewest episodes, but the smallest network [16] reaches the convergence threshold fastest in wall-clock time because each training step is cheaper. The largest network [256, 256] is slowest per step and has the highest seed-to-seed variance in convergence episodes.

2 Introduction

2.1 Motivation

Autonomous vehicles must choose throttle, brake, and steering many times per second. Good control is not a single decision — it depends on what happened earlier in the lap. For example, too much speed before a corner can push the car off the track later.

Reinforcement learning (RL) is well suited to this kind of problem. An agent interacts with an environment, receives rewards, and learns a policy over many trials. No full driving model needs to be programmed by hand.

This project applies RL to a **simplified 2D car racing task**. One car must drive around a **closed track** as fast as possible while staying on the road. The setup is simpler than a real car simulator, but it still requires sequential decision-making, continuous control, and a clear performance goal.

2.2 Goal

The training objective is to **complete one full lap in minimum time**.

A reference racing line is **not** hand-coded. The agent must learn when to accelerate, brake, and steer from the reward signal alone. The policy is trained with **Proximal Policy Optimization (PPO)** and a neural network that maps observations to actions.

The full pipeline was implemented in Python: track generation, vehicle dynamics, a custom **Gymnasium** environment, training with **Stable-Baselines3**, evaluation, and result plots.

2.3 Research questions

The RL algorithm and hyperparameters are kept **identical** across all runs. The only variable is **policy network size** — the number of neurons in the hidden layers. Three sizes are compared: [16], [64, 64], and [256, 256].

The project addresses three questions:

1. **Final performance** — After training, how well does the agent drive? This is measured by evaluation return, lap completion rate, and lap time on a completed lap.
2. **Episodes to convergence** — How many training episodes are needed before performance stabilises? Convergence is defined as the first episode where a 50-episode rolling-mean return reaches 90% of the run’s final rolling mean.
3. **Wall-clock time** — How long does training take in real seconds on the training machine? This matters for practical cost, not only learning speed in episodes.

Each network size is trained with **four random seeds** (0, 1, 2, 3). Results are reported as **mean $\pm$ standard deviation** across seeds. All experiments use a **fixed track** so that differences come from network size, not from changing road geometry.

3 Background

3.1 Reinforcement learning basics

An RL agent interacts with an **environment** over many **steps**. At each step:

1. The agent observes state s_t .
2. It chooses action a_t .
3. The environment returns reward r_t and the next state s_{t+1} .

The agent’s goal is to maximize the **total return** (sum of rewards) over an **episode** — one run from start to finish.

3.2 Policy gradient methods

A **policy** is a function $\pi(a \mid s)$ that maps states to actions. In **policy gradient** methods the policy is represented with a neural network and its weights are updated to increase expected return.

This project uses **continuous actions** (throttle and steering), so the policy outputs a **Gaussian distribution** over actions. During training, actions are **sampled** from that distribution to explore. During evaluation, the **mean** action is used only.

3.3 PPO (Proximal Policy Optimization)

PPO is a popular on-policy algorithm. It updates the policy many times on recent data, but limits how much the policy can change in one update. This makes training more stable than vanilla policy gradients.

PPO from **Stable-Baselines3** is used with an MLP policy (**MlpPolicy**). The same network architecture is used for both the **actor** (actions) and the **critic** (value estimate).

3.4 Why this problem fits RL

- The task is sequential: early mistakes (e.g. too much speed before a corner) cause later failure.
- The reward depends on progress along the track, not on a single correct answer.
- The agent must balance speed and safety from experience.

4 Environment Design

4.1 Track generation

The track is a **closed loop**. First, 8–12 control points are placed in the plane. Then a **periodic cubic spline** is fitted through them, and the centerline is resampled at uniform **arc-length** spacing. Each resampled point stores:

- cumulative arc length s
- unit tangent vector
- signed curvature κ (positive = left turn, negative = right turn)

The road has constant half-width (6 m). A car is on-track when its lateral offset $|d| \leq$ half-width.

Figure 1 shows example tracks used in the project (fixed Monaco-style layout and random variants).

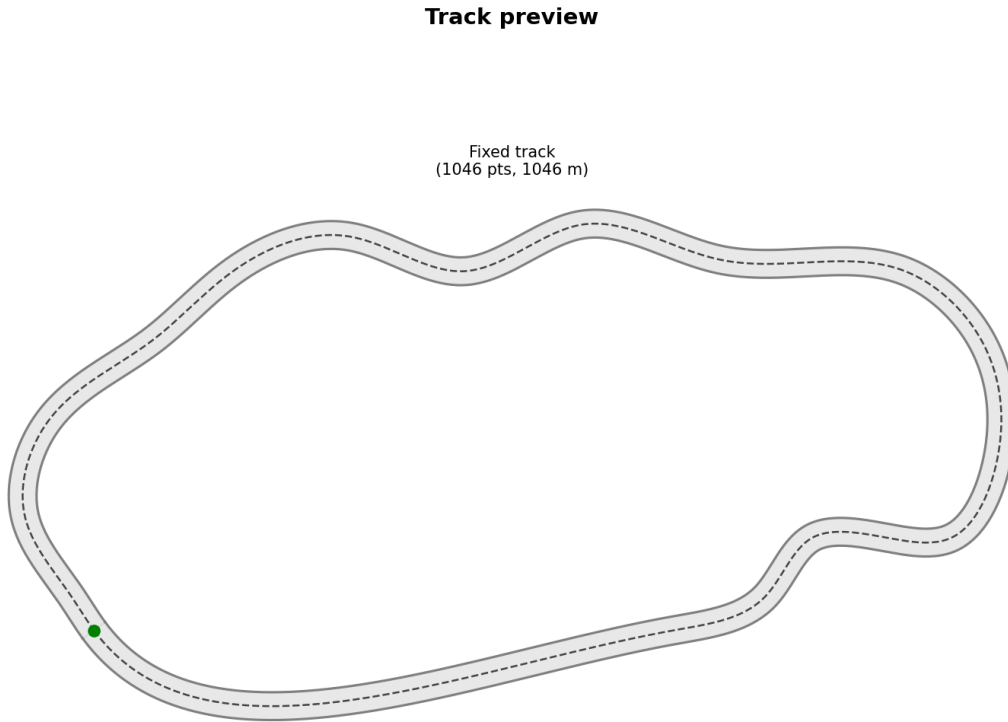


Figure 1: Example racetracks generated by the project (fixed and random modes).

How to read this figure: Each panel is a top-down view of a track centerline. The agent always sees the road relative to itself (Frenet frame), not absolute (x, y) position. If tracks are regenerated, the shapes may change but the filename stays `tracks_preview.png`.

4.2 Car dynamics

A **kinematic bicycle model** is used with forward Euler integration and $dt = 0.05$ s (20 Hz). Main constants:

Parameter	Value
Max speed $v_{\max}$	30 m/s
Max acceleration	8 m/s ²
Max steering	0.45 rad
Wheelbase L	3.0 m

Given acceleration a and steering δ , the model updates position, heading, and speed. Speed is clipped to $[0, v_{\max}]$.

4.3 Frenet coordinates

Cartesian position (x, y) is hard for the agent to interpret. Positions are therefore converted to **Frenet** track-relative coordinates:

Symbol	Meaning
s	Arc length along centerline from the start
d	Lateral offset from centerline (+ left, - right)
θ_e	Heading error vs. track tangent

Lookahead curvatures are also computed at distances $[5, 10, \dots, 110]$ m ahead so the agent can anticipate corners.

Design choice: s is **not** included in the observation. Absolute progress does not generalize well across different tracks. Offset, heading error, speed, and upcoming curvature do.

4.4 Observation space (14 numbers)

All features are normalized to roughly $[-1, 1]$:

Index	Feature
0	$d/\text{half_width}$
1–2	$\sin(\theta_e), \cos(\theta_e)$
3	$v/v_{\max}$
4–13	Signed curvature at 10 lookahead distances $\times$ scale factor

4.5 Action space (2 numbers)

Both actions lie in $[-1, 1]$:

- `action[0]` $\rightarrow$ acceleration = $8.0 \times \text{action}[0]$ m/s²
- `action[1]` $\rightarrow$ steering = $0.45 \times \text{action}[1]$ rad

4.6 Reward function

Each step:

$$\text{reward} = 1.0 \times \Delta s - 0.01$$

Event	Effect
Progress Δs	Reward for moving forward along the track
Time penalty -0.01	Small cost per step to discourage stopping
Off-track ($\ d\ > \text{half-width}$)	-80 penalty, episode terminated
Full lap completed	$+100$ bonus, episode terminated
2000 steps reached	Episode truncated (not terminated)

Gymnasium separates **terminated** (real end: crash or lap done) from **truncated** (time limit). PPO uses this distinction when bootstrapping value targets.

5 Training and Evaluation

5.1 Network architectures

Three hidden-layer sizes are tested (same hyperparameters otherwise):

Label	Hidden layers	Role in study
Tiny	[16]	Very low capacity
Small	[64, 64]	Medium
Large	[256, 256]	High

5.2 Training procedure

- Algorithm: PPO (**stable-baselines3**), default hyperparameters unless noted in `config.json`
- Parallel envs: 8 subprocess environments (**SubprocVecEnv**)
- Total training steps: 1,000,000 per run (configurable via CLI)
- Seeds: 0, 1, 2, 3 per architecture
- Track mode for experiments: **fixed** (same track every episode)

Each episode is logged to `progress.csv`: return, length, mean log std of the Gaussian policy, wall-clock time.

5.3 Evaluation procedure

During training, actions are **stochastic** (sampled). For evaluation, **deterministic=True** is set and the Gaussian **mean** μ is always used. This measures what the policy actually learned, without exploration noise.

Metrics:

- **Mean return** — average total reward over evaluation episodes
- **Completion rate** — fraction of episodes that finish a full lap
- **Lap time** — steps (and simulated seconds) for completed laps

6 Experiments and Results

A grid of **3 architectures** $\times$ **4 seeds** = **12 training runs** was executed. After training, `python -m src.plots` builds all figures below into `experiments/figures/`. **Filenames are fixed**; if experiments are re-run, the images and table update but the report references stay the same.

Convergence definition (used in Figures 3–4): first episode where the 50-episode rolling-mean return reaches **90%** of that run’s own final 50-episode rolling mean.

6.1 Learning curves

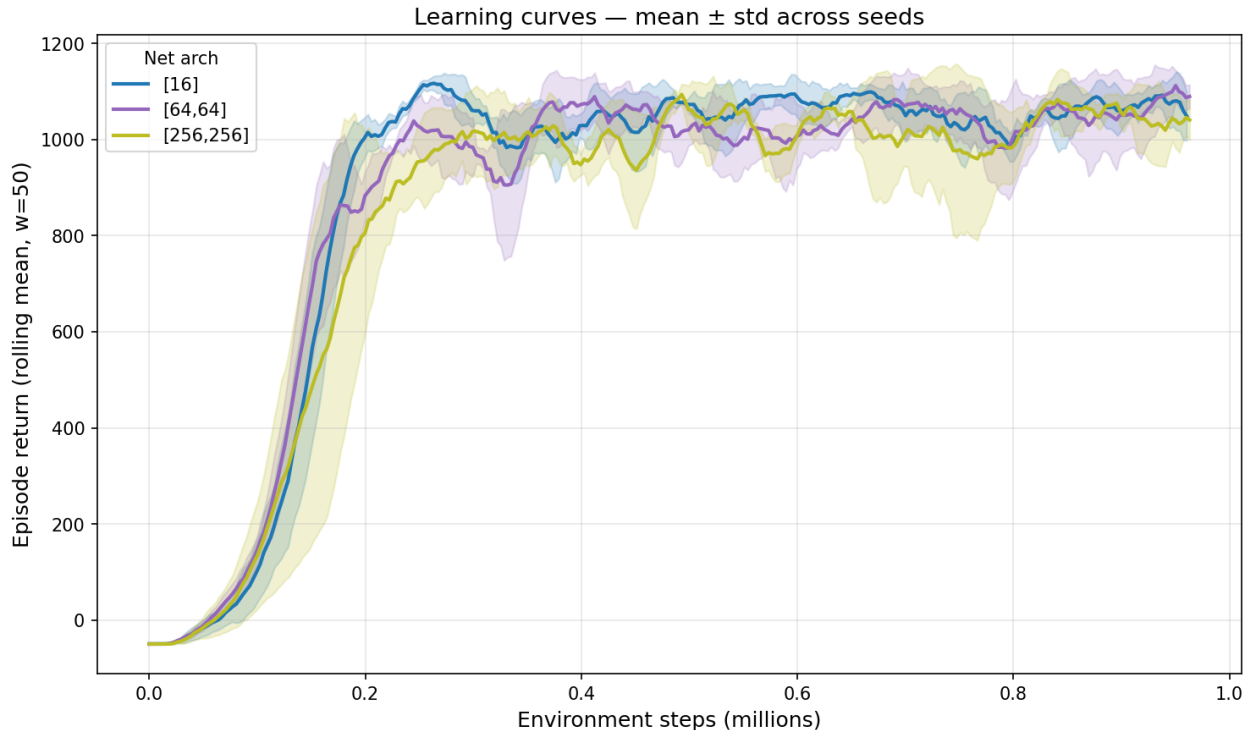


Figure 2: Rolling-mean episode return vs environment steps (mean $\pm$ std across seeds).

How to read: The x-axis is cumulative environment steps. The y-axis is episode return, smoothed with a 50-episode rolling mean. Each colour is one network size; shaded bands show variability across the four seeds.

Results: All three architectures show a clear upward trend and eventually reach returns near 1100–1140 (a completed lap). The [64, 64] and [256, 256] curves rise faster in the early phase; [16] learns more slowly at first but catches up before the end of training. Final performance is very similar across sizes — extra capacity does not produce a much higher return on this fixed track.

6.2 Final performance

How to read: Bars show **deterministic** evaluation after training: mean return and lap completion rate, averaged over four seeds (error bars = std across seeds).

Results: Every architecture reaches **100% completion** on all seeds. Mean return is about **1139** for all three (differences under 1 point). Lap time is fastest for [64, 64] ($\sim$ **35.6 s** simulated) and

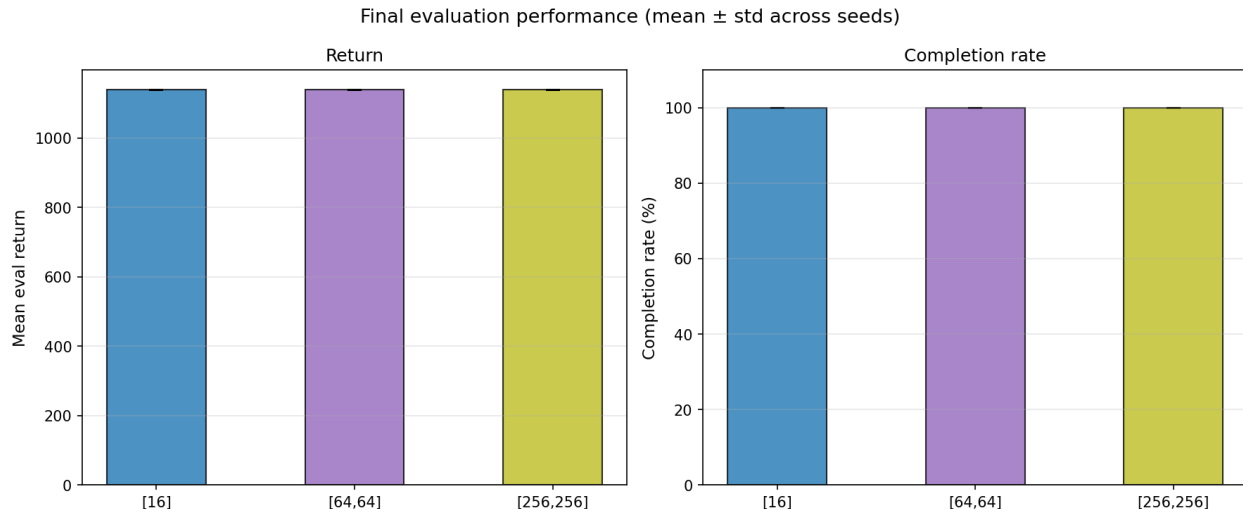


Figure 3: Evaluation return and lap completion rate per network size.

slightly slower for [16] (~ 36.3 s). Once the network is large enough to represent the policy, **bigger does not mean faster laps** on this task.

6.3 Convergence in episodes

How to read: Lower bars mean the rolling-mean return reached 90% of its final value in fewer training episodes.

Results: [64, 64] converges in the **fewest episodes** (284 ± 52), followed by [256, 256] (326 ± 132) and [16] (333 ± 23). The trend is **not monotonic** with network size — the medium network is the most sample-efficient here. [256, 256] has the **largest spread across seeds** (± 132 episodes), so training is less predictable despite high final performance.

6.4 Convergence in wall-clock time

How to read: Same convergence rule as above, but measured in **real seconds** on the training machine.

Results: Wall-clock order differs from the episode ranking: [16] is fastest (115 ± 8 s), then [64, 64] (126 ± 25 s), then [256, 256] (178 ± 63 s). Each extra layer costs more compute per step (see table: 0.64–0.79 s per 1k steps). A network that converges in fewer episodes can still take **longer in real time** if each step is slower.

6.5 Policy exploration (sigma decay)

How to read: `mean_log_std` measures how much randomness remains in the Gaussian policy. It typically **decreases** as the agent becomes more confident.

Results: All architectures show exploration decaying over training — the agent gradually commits to sharper actions. Curves have a similar shape; there is no sign that larger networks keep exploring much longer. By the end of training, all policies operate with low action noise in evaluation mode.

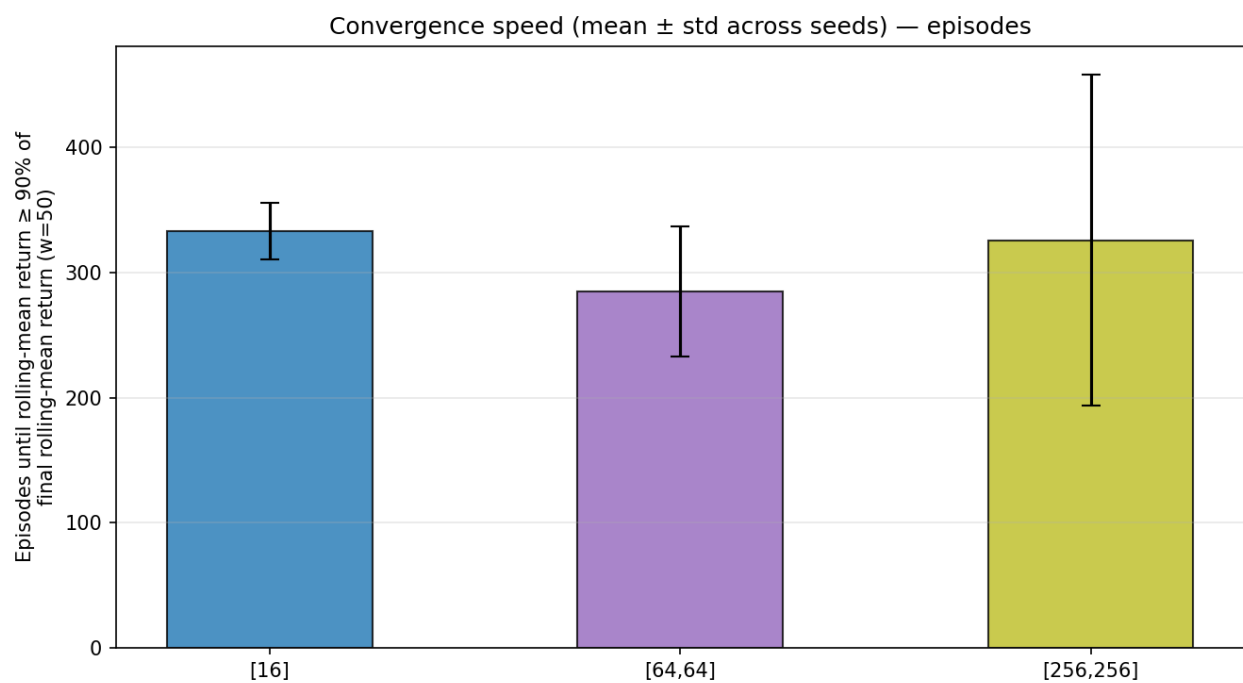


Figure 4: Episodes until convergence criterion is met (mean $\pm$ std).

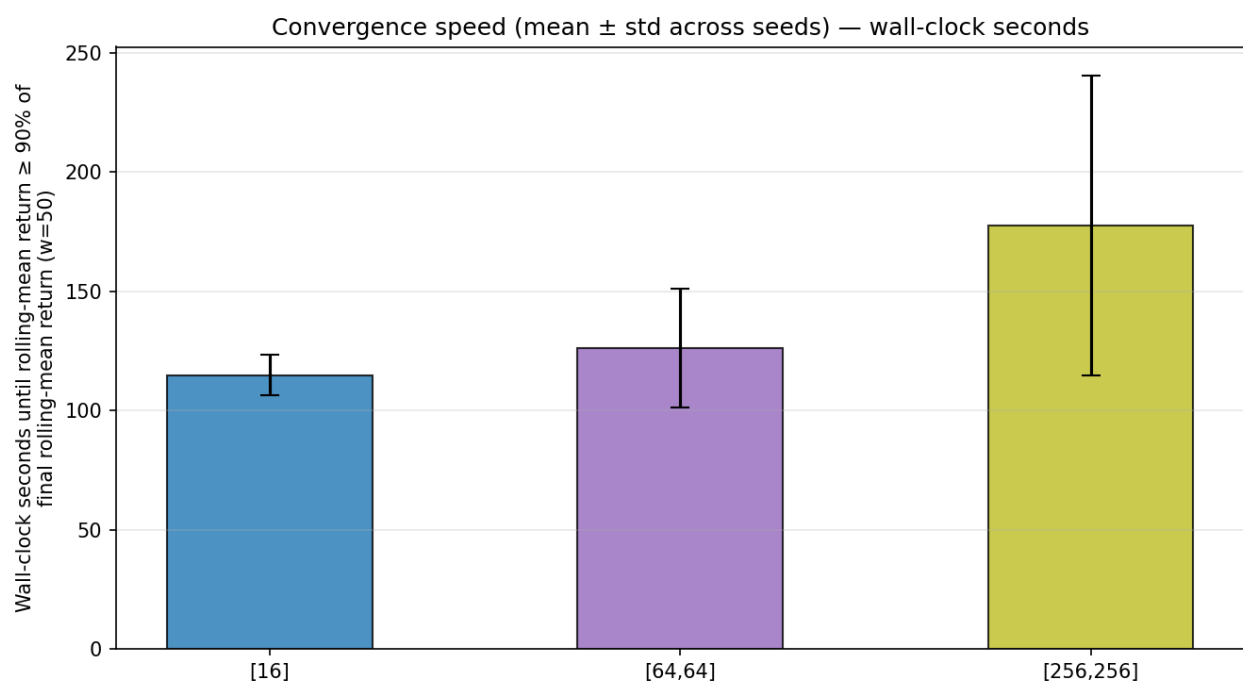


Figure 5: Wall-clock seconds until convergence (mean $\pm$ std).

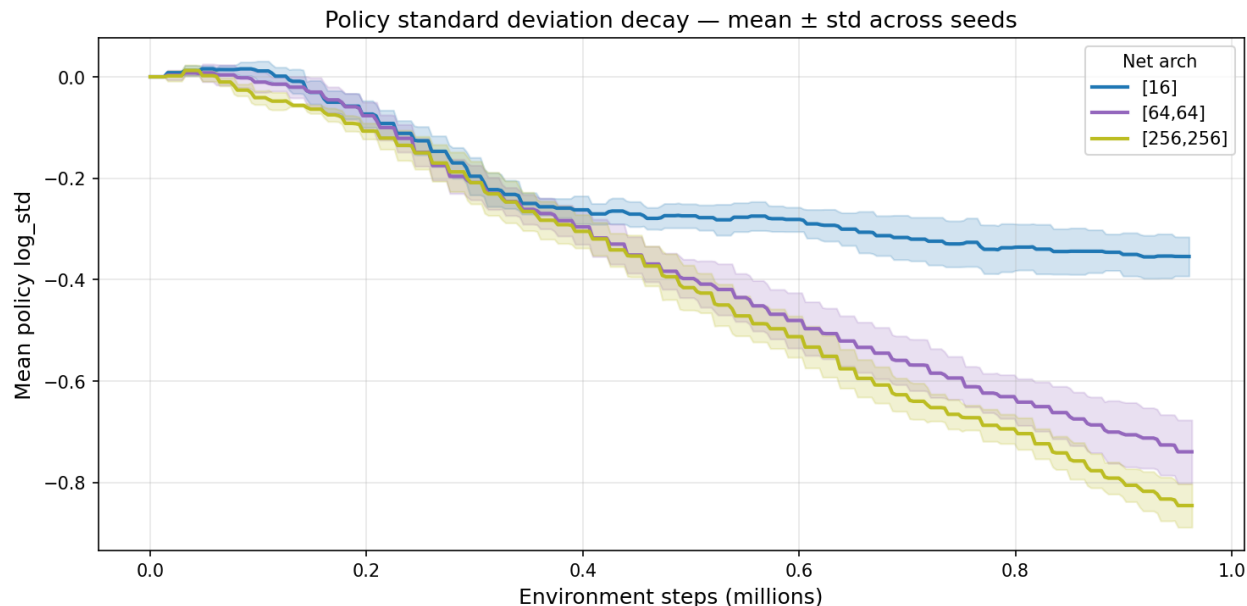


Figure 6: Policy log-standard-deviation vs training steps.

6.6 Summary table

The table below is copied from `experiments/figures/results_table.md` (12 runs: 3 architectures $\times$ 4 seeds).

Convergence = first episode where 50-episode rolling-mean return $\geq 90\%$ of the run’s own final 50-episode rolling mean.

Net arch	Final return	Completion	Lap time (steps)	Lap time (sim s)	Episodes to converge	Wall-clock to converge (s)	Wall-clock / 1k steps (s)
[16]	1139.2 $\pm$ 0.5	100.0%	725	36.25 s	333 $\pm$ 23	115 $\pm$ 8	0.636 $\pm$ 0.008
[64,64]	1139.4 $\pm$ 0.5	100.0%	711	35.56 s	284 $\pm$ 52	126 $\pm$ 25	0.686 $\pm$ 0.003
[256,256]	1139.4 $\pm$ 0.5	100.0%	712	35.59 s	326 $\pm$ 132	178 $\pm$ 63	0.786 $\pm$ 0.041

How to read the table: All rows complete the lap. Return and lap time are nearly identical across sizes; [64, 64] has the best lap time. For convergence, compare episodes (sample efficiency) with wall-clock (compute cost) — they rank architectures differently.

7 Discussion

7.1 Network size vs performance

All three tested sizes — including the tiny [16] network — learn to complete the lap with **100% success** and similar return (~1139). Final lap time differs by only about **0.7 simulated seconds** between the fastest ([64, 64], 35.6 s) and the slowest ([16], 36.3 s). For this fixed track and observation space, **extra parameters do not improve driving quality** once the network is large enough.

7.2 Network size vs training cost

Training cost tells a different story than final performance:

- **Episodes:** [64, 64] converges in the fewest episodes (284), so it is the most sample-efficient.
- **Wall-clock:** [16] reaches convergence in the shortest real time (115 s) because each step is cheapest (0.64 s / 1k steps).
- **Variance:** [256, 256] has the widest spread in convergence episodes (± 132), making it the least predictable across seeds.

Bigger networks are **not** faster to train in practice here, even when they sometimes converge in fewer episodes than [16].

7.3 Limitations

- **Simplified physics** — kinematic bicycle, no tire slip dynamics.
- **Fixed track** — all runs use the same layout; results may not transfer to random tracks without retraining.
- **Single machine** — wall-clock times depend on CPU/GPU and parallel env count.
- **Narrow performance range** — all successful policies achieve nearly the same return, so differences are mainly in training cost, not final lap quality.

7.4 Reproducibility

```
python -m src.train --net-arch 64,64 --seed 0 --total-steps 1000000
python -m src.evaluate --run-name arch64_64_seed0_fixed --n-episodes 20
python -m src.plots
```

8 Conclusion

A custom Gymnasium racing environment was built with Frenet observations and PPO training. The effect of policy network size on driving performance and training cost was studied over 12 runs (3 architectures $\times$ 4 seeds, 1M steps each).

1. **Final performance** — All three networks complete the lap on every seed with mean return ~ 1139 . [64, 64] has the fastest lap time (~ 35.6 s); [16] is only slightly slower (~ 36.3 s). Network size barely affects final driving quality on this task.
2. **Episodes to convergence** — [64, 64] converges in the fewest episodes (284 ± 52). [16] and [256, 256] need more episodes (333 and 326 on average). Bigger is not always faster to learn.
3. **Wall-clock time** — [16] reaches convergence in the shortest real time (115 ± 8 s) because it is fastest per training step. [256, 256] is slowest (178 ± 63 s). For a fixed compute budget, a smaller network can be the better choice even if it needs more episodes.

Overall, for this racing task the **medium network** [64, 64] offers the best balance: fastest lap, fewest episodes to converge, and only slightly more wall-clock time than [16]. The largest network adds compute cost without improving final performance.